



UNIVERSIDAD
CATÓLICA
BOLIVIANA

Segunda Evaluación: Portal SWAPI

Autor: Andrés Ignacio Hinojosa Garnica

Universidad: Universidad Católica Boliviana – Facultad de Ingeniería

Curso: Tecnologías Web

Versión: 1.0.0

Índice

1. Resumen
2. Demostración Funcional
3. Arquitectura y Diseño
4. Implementación Técnica
5. Consumo de la API SWAPI
6. Calidad y Mantenimiento

1. Resumen

Este documento presenta el **Portal SWAPI**, una aplicación web desarrollada con **Vue 3** y **Vite** que consume la API pública de Star Wars (SWAPI). El sistema ofrece:

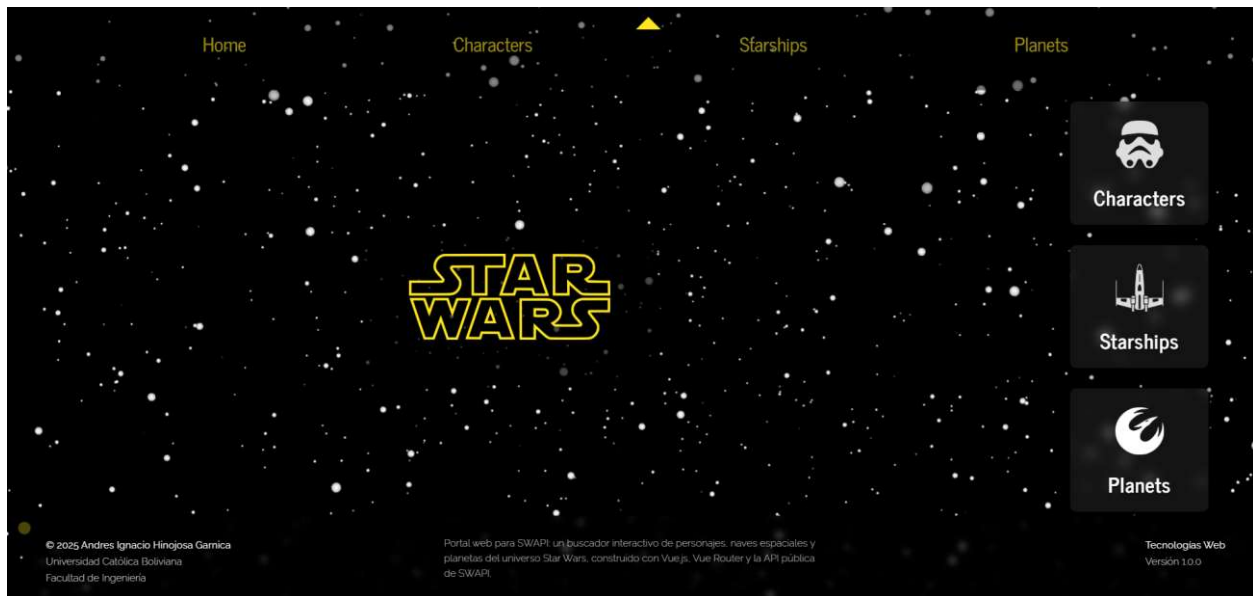
- **Listados interactivos** de personajes, naves espaciales y planetas con paginación infinita.
- **Módulos reutilizables** (CharacterCard, StatusIndicator, ErrorBoundary, etc.) para facilitar la extensibilidad.
- **Gestión de errores y loading** con ESLint + Prettier, asegurando un flujo de desarrollo limpio y controlado.

A continuación se detalla la demostración funcional, el diseño arquitectónico, aspectos de implementación, consumo de la API y las prácticas de calidad adoptadas.

2. Demostración Funcional

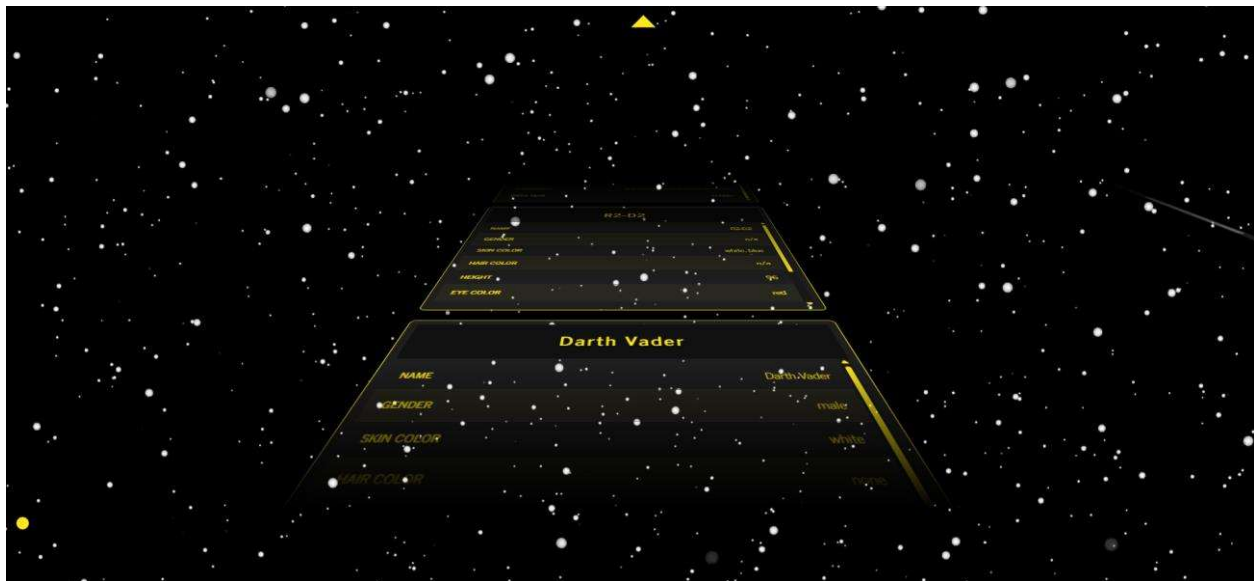
Inicio de la Aplicación

1. **Pantalla de Bienvenida:** Logotipo temático y accesos visuales a secciones clave.



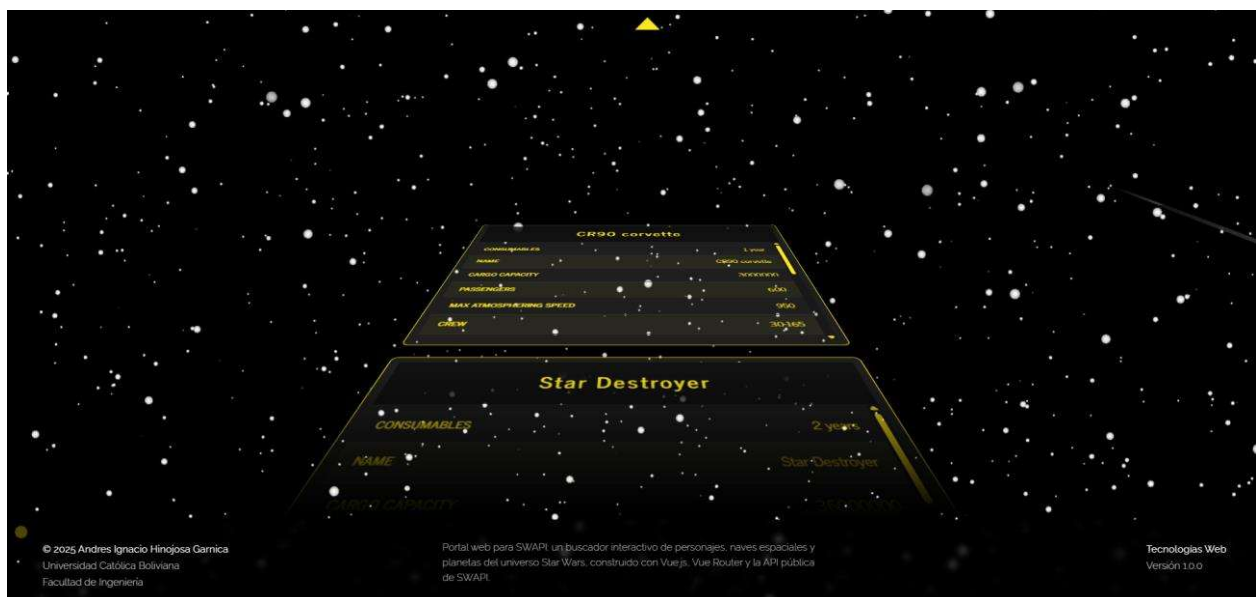
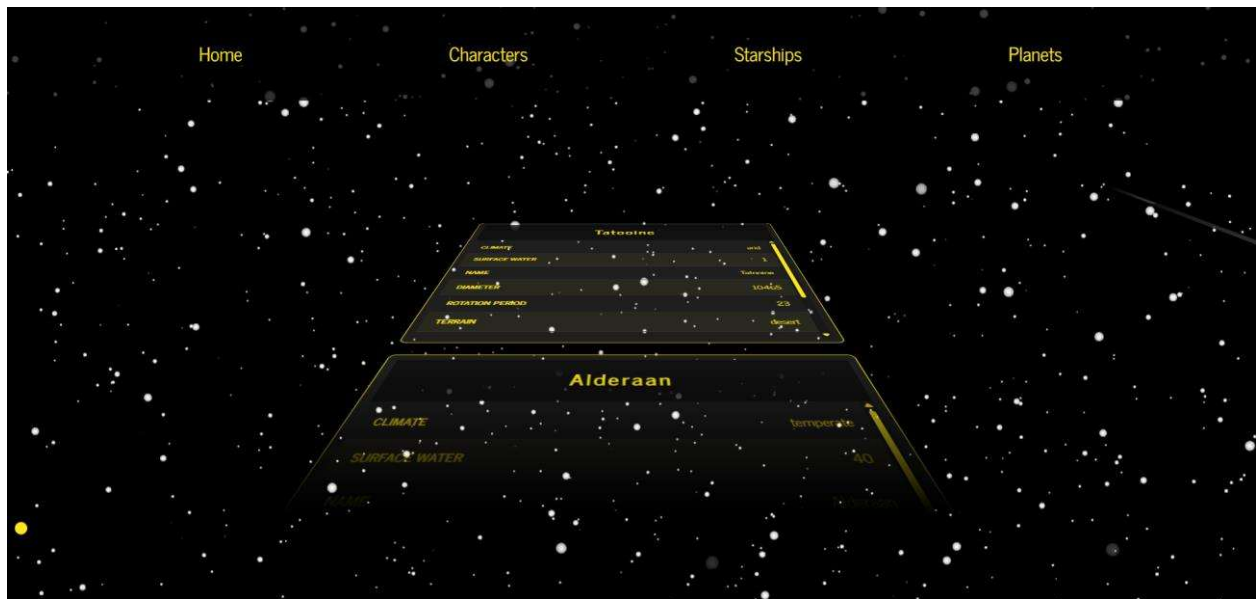
Listado de Personajes (Infinite Scroll)

1. Despliegue automático de nuevos registros al llegar al final de la lista.
2. Indicador de carga centralizado y manejo de errores en fallos de red.



Naves Espaciales y Planetas

- **StarshipList.vue** y **PlanetList.vue** replican la lógica de scroll infinito.
- Validación de datos y presentación con tarjetas estilizadas.



3. Arquitectura y Diseño

Estructura de Carpetas

```
frontend/
├── src/
│   ├── components/
│   │   └── Navbar.vue
│   └── animaciones
```

Contenedor global de navegación con slot para ítems y

```

├── CharacterCard.vue # Tarjeta de atributos genéricos (personaje, nave, planeta)
├── StatusIndicator.vue # Overlay de carga y errores con backdrop-filter
├── ErrorBoundary.vue # Captura de excepciones en renderizado de tarjetas
├── Footer.vue # Pie de página emergente con diseño en grid
├── views/
│   ├── CharacterList.vue # Vista de personajes con infinite scroll
│   ├── StarshipList.vue # Vista de naves espaciales (reusa lógica de scroll)
│   └── PlanetList.vue # Vista de planetas
├── router/
│   └── index.js # Definición de rutas y configuración de history
├── App.vue # Componente raíz que carga Navbar, RouterView y Footer
├── docs/ # Imágenes y capturas para documentación
├── eslint.config.cjs # Configuración Flat de ESLint v9+ para JS y Vue
└── vite.config.js # Configuración de Vite con plugins y alias

```

Diagrama de Componentes y Flujo

```

App.vue
├── Navbar.vue
├── RouterView
│   ├── CharacterList.vue
│   │   ├── ErrorBoundary.vue
│   │   │   └── CharacterCard.vue
│   ├── StarshipList.vue
│   │   ├── ErrorBoundary.vue
│   │   │   └── CharacterCard.vue
│   └── PlanetList.vue
│       ├── ErrorBoundary.vue
│       │   └── CharacterCard.vue
└── Footer.vue

```

Descripción de Componentes

- **Navbar.vue:** implementa la barra de navegación oculta, con animación de deslizamiento y un indicador sutil para descubrirla. Expone un `<slot>` donde se inyectan enlaces de `router-link` desde `App.vue`.
- **CharacterCard.vue:** componente genérico que recibe un objeto `properties` y renderiza dinámicamente cada campo con estilo temático. Usa un fondo traslúcido y `backdrop-filter: blur()` para simular un panel holográfico.

- **StatusIndicator.vue**: overlay de pantalla completa durante peticiones HTTP. Bloquea interacciones y aplica backdrop-filter: blur(5px). Muestra spinner y mensajes de error condicionales.
- **ErrorBoundary.vue**: wrapper que captura errores en el render de cada tarjeta y utiliza el slot de fallback (StatusIndicator) para mostrar fallos sin romper toda la vista.
- **Footer.vue**: pie de página emergente en tres columnas, con diseño grid y un indicador circular animado. Se expande al hacer hover mostrando ©, datos académicos y descripción del proyecto.
- **CharacterList.vue, StarshipList.vue, PlanetList.vue**: vistas que implementan paginación infinita. Comparten lógica: mantienen un ref de nextUrl, usan fetch() en loadMore() y un IntersectionObserver sobre un sentinel para desencadenar nuevas cargas.
- **router/index.js**: configura rutas “/characters”, “/starships” y “/planets”, usa createWebHistory y carga lazy las vistas para optimizar el bundle.

4. Implementación Técnica

Vite y Plugins

- **vite.config.js** carga @vitejs/plugin-vue y vite-plugin-eslint para lint continuo.
- Alias @ apuntando a src/ para importaciones absolutas.

```
import eslintPlugin from 'vite-plugin-eslint'
export default defineConfig({
  plugins: [vue(), eslintPlugin({ include: ['src/**/*.vue', 'src/**/*.js'] })]
})
```

Routing

```
import { createRouter, createWebHistory } from 'vue-router'
const routes = [
  { path: '/characters', component: CharacterList },
  { path: '/starships', component: StarshipList },
  { path: '/planets', component: PlanetList }
]
const router = createRouter({ history: createWebHistory(), routes })
```

Infinite Scroll y Estado

1. **IntersectionObserver** monitoriza un `<div ref="sentinel">`.
2. **Estado Reactivo** (ref, onMounted) recarga lotes cuando el sentinel entra en viewport.

```
const characters = ref([])
const nextUrl = ref('https://www.swapi.tech/api/people')
function loadMore() { ... }
onMounted(() => observer.observe(sentinel.value))
```

5. Consumo de la API SWAPI

Endpoints Principales

- **Personajes:** GET /api/people?page=N
- **Naves:** GET /api/starships?page=N
- **Planetas:** GET /api/planets?page=N

Flujo de Datos

1. **Listar** → obtiene lista con enlaces a detalles.
2. **Detalle** → solicita `fetch(detail.url)` para propiedades completas.
3. **Unión** → agrega las propiedades al arreglo principal.

Manejo de errores: try/catch alrededor de fetch, con `StatusIndicator.vue` mostrando mensajes claros.

6. Calidad y Mantenimiento

ESLint con Flat Config (v9+)

```
// eslint.config.cjs
const js = require('@eslint/js')
const pluginVue = require('eslint-plugin-vue')
const eslintPrettier = require('eslint-config-prettier')
module.exports = [
```

```
js.configs.recommended,  
...pluginVue.configs['flat/recommended'],  
{ files:['**/*.js,vue'], languageOptions:{ ecmaVersion:'latest', sourceType:'module',  
globals:{...} }, rules:{  
  'vue/multi-word-component-names':'off', 'no-console':'warn' }  
},  
eslintPrettier  
]
```

Formateo con Prettier

- Archivo .prettierrc especifica comillas simples y sin punto y coma.
- Integrado en el pipeline de ESLint para evitar conflictos.